

SPRING 2022 SOLVE
CSE-1221
Computer Programming II

[© REZOWAN SIFAT](#)

1. a) Define operator overloading? Write the rules of operator overloading.

Ans: Operator overloading is a feature in C++ that allows you to redefine the behavior of an operator for a specific class or data type

The rules for operator overloading in C++ are as follows:

1. Overloaded operators must have at least one operand that is a user-defined type (class object or enumeration).
2. You cannot create new operators or change the precedence of existing operators. Only the behavior of existing operators can be modified.
3. You cannot overload certain operators such as `sizeof`, `::` resolution operator, `.`, `.*`, and `?:`.
4. The overloaded operator must be a non-static member function or a global function. It cannot be a static member function.
5. Overloaded operators should be implemented with care to maintain logical consistency and avoid unexpected behavior.

Example To Understand The Concept Of Operator Overloading

```
#include <iostream>
using namespace std;

class Complex {
private:
    int real, imag ;

public:
    Complex () {
        real = 0;
        imag = 0;
    }
}
```

```

        Complex (int r, int i){
            real = r;
            imag = i;
        }

        void print (){
            cout<<real<<" + "<<imag<<endl;
        }

        Complex operator +(Complex c){
            Complex temp;
            temp.real =real + c.real;
            temp.imag = imag + c.imag;

            return temp;
        }
};

int main () {
    Complex c1(5,4);
    Complex c2(2,5);
    Complex c3(1,1);
    Complex c4;
    c4 = c1+c2+c3; //c1.add(c2);

    c4.print();

    return 0;
}

```

1.b) Create a class float that contains one float data member. Overload an arithmetic operator so that it can operate on the objects of float.

```

#include <iostream>
using namespace std;

class Float
{
private:
    float value;

```

```

public:
    Float(float floatValue)
    {
        this->value = floatValue;
    }
    Float()
    {
        value = 0;
    }

    Float operator+(Float f)
    {
        Float temp;
        temp.value = value + f.value;
        return temp;
    }

    void getter()
    {
        cout << value << endl;
    }
};

int main()
{
    Float f1(2.5);
    Float f2(3.7);
    Float f3;

    f3 = f1 + f2;
}

```

```
f3.getter();  
  
return 0;  
}
```

Or Construct a program to overload logical AND (&&) operator using friend function.

```
#include <iostream>  
using namespace std;  
  
class Logic  
{  
private:  
    int value;  
  
public:  
    Logic(int val)  
    {  
        this->value = val;  
    }  
  
    friend bool operator&&(Logic l1, Logic l2)  
    {  
        return l1.value && l2.value;  
    }  
  
    void getter()  
    {  
        cout << value << endl;  
    }  
};
```

```

int main()
{
    Logic obj1(0);
    Logic obj2(10);

    if (obj1 && obj2)
    {
        cout << "Both objects evaluate to true." <<
endl;
    }
    else
    {
        cout << "At least one object evaluates to
false." << endl;
    }

    return 0;
}

```

1.c) A friend function cannot be used to overload the assignment operator (=) Explain why?

ANS: A friend function cannot be used to overload the assignment operator (`=`) because the assignment operator is a special member function that is automatically generated by the compiler if it is not explicitly defined. It is used to assign the value of one object to another of the same class.

The assignment operator is typically defined as a member function of the class itself and has specific requirements and behavior. It needs to be able to access the private members of the class to perform the assignment correctly.

On the other hand, a friend function is a non-member function that is granted access to the private members of a class. It is typically used to provide external functionality that needs access to the private members of the class.

However, when it comes to the assignment operator, it is already a member function and is implicitly declared as a friend of the class. This allows it to access the private members of the class directly, without the need for explicit friend declaration.

Therefore, there is no need to use a friend function to overload the assignment operator because the assignment operator is already a member function with the necessary privileges to access the private members of the class.

1.d) Answer the following question with necessary example and explanation

i) When an operator is overloaded, does it lose any of its original functionality?

ANS: When an operator is overloaded, it does not lose any of its original functionality. The overloaded operator retains its original meaning and behavior for the built-in types. However, for user-defined types, the overloaded operator can have a different implementation based on the specific needs of the class.

For example, let's consider the addition operator (+). By default, the addition operator performs addition of numerical values. When you overload the addition operator for a custom class, it can still perform addition, but with a different implementation that makes sense for that class. The original functionality of the addition operator for built-in types remains intact.

Here's an example to illustrate this:

```
#include <iostream>
using namespace std;

class Complex {
private:
    int real, imag ;

public:
    Complex () {
        real = 0;
        imag = 0;
    }

    Complex (int r, int i){
        real = r;
        imag = i;
    }

    void print () {
        cout<<real<<" + "<<imag<<endl;
    }

    Complex operator +(Complex c){
        Complex temp;
        temp.real =real + c.real;
        temp.imag = imag + c.imag;

        return temp;
    }
};

int main () {
    int a = 5, b = 10;
    int sum1 = a + b;

    Complex c1(5,4);
    Complex c2(2,5);
    Complex c3(1,1);
    Complex c4;
    c4 = c1+c2+c3;

    c4.print();

    return 0;
}
```

**Or II) Can the precedence of an overloaded operator be changed?
Can the number of operand be altered?**

ANS: The precedence of an overloaded operator cannot be changed. The precedence and associativity of operators in C++ are determined by the language rules and cannot be altered when overloading operators.

Similarly, the number of operands for an operator cannot be altered when overloading it. The arity (number of operands) of operators is fixed and cannot be changed during operator overloading. For example, the binary addition operator (+) always takes two operands, and overloading it will also involve two operands.

Overloading operators allows us to define custom behavior for operators with user-defined types, but the basic rules and characteristics of the operators, including precedence and arity, remain unchanged.

2.a) "A derived class can access all the members of its base class."-Is this statement true?

Yes, the statement is true. In C++, a derived class can access all the members of its base class, including public, protected, and private members.

Here's a brief explanation of member accessibility in C++ inheritance:

1. Public members: Public members of the base class are accessible to the derived class.
2. Protected members: Protected members of the base class are also accessible to the derived class. They can be accessed in the same way as public members.
3. Private members: Private members of the base class are not directly accessible to the derived class. However, they are still present in the derived class and can be accessed through public or protected member functions of the base class.

It's important to note that the accessibility of members in a derived class depends on the type of inheritance used. In public inheritance, both public and protected members of the base class become public and protected members of the derived class, respectively. In private

inheritance, both public and protected members of the base class become private members of the derived class.

or, In inheritance, explain why the protected category is needed?

The protected access specifier in inheritance serves as an intermediate level of access control between public and private.

The protected category is needed in inheritance for several reasons:

1. Encapsulation: Protected members allow for encapsulation within the inheritance hierarchy. They provide a level of data and functionality that is not directly accessible to external classes but can be accessed by derived classes. This promotes the principle of information hiding and helps maintain the integrity of the base class.
2. Frameworks and libraries: In larger projects, frameworks, and libraries, protected members can be used to expose certain functionality to derived classes while still hiding implementation details from the outside world. This helps in maintaining a well-defined API for the framework or library, allowing for easier extension and customization.
3. Polymorphism: The protected access specifier is also essential for achieving polymorphism, which is a fundamental concept in object-oriented programming. Without protected members, derived classes wouldn't have access to the base class's protected members, limiting their ability to participate fully in polymorphic behavior.

2.b) Write a program to implement multiple inheritance.

```
#include <iostream>

using namespace std;

class Base1{
```

```
public:

    void display1() {

        cout << "Base1 Display" << endl;

    }

};

class Base2{
public:

    void display2() {

        cout << "Base2 Display" << endl;

    }

};

class Derived : public Base1 , public Base2{

    public:

    void displayDerived() {

        cout << "Derived Display" << endl;

    }

};

int main()
```

```
{  
  
    Derived dobj;  
  
    dobj.display1();  
    dobj.display2();  
    dobj.displayDerived();  
  
    return 0;  
}
```

Or

What is virtual base class? Explain with writing a program.

The virtual base class is a concept used in multiple inheritances to prevent ambiguity between multiple instances.

```
#include <iostream>  
  
using namespace std;  
  
// Virtual base class  
  
class Base  
{
```

```
public:

    int baseValue;

    virtual void display()

    {

        cout << "Base Display: " << baseValue << endl;

    }

};

// Derived class 1

class Derived1 : virtual public Base

{

public:

    int derived1Value;

    void display() override

    {

        cout << "Derived1 Display: " << derived1Value << endl;

    }

};

// Derived class 2

class Derived2 : virtual public Base
```

```
{  
  
public:  
  
    int derived2Value;  
  
    void display() override  
    {  
        cout << "Derived2 Display: " << derived2Value << endl;  
    }  
};  
  
// Derived class inheriting from both Derived1 and Derived2  
class DerivedCombined : public Derived1, public Derived2  
{  
public:  
    int combinedValue;  
  
    void display() override  
    {  
        cout << "DerivedCombined Display: " << combinedValue  
<< endl;  
    }  
};
```

```
int main()
{
    DerivedCombined derivedObj;

    derivedObj.baseValue = 10;

    derivedObj.derived1Value = 20;

    derivedObj.derived2Value = 30;

    derivedObj.combinedValue = 40;

    derivedObj.display();

    return 0;
}
```

2.c) How to invoke Base class's parameterized constructor inside Derived class's parameterized constructor? Give an example.

To invoke a base class's parameterized constructor inside a derived class's parameterized constructor, you can use the initialization list of the derived class. The initialization list allows you to specify the constructor of the base class to be invoked with the required arguments.

```
#include <iostream>

using namespace std;
```

```
// Base class

class Base
{
public:

    int baseValue;

    // Parameterized constructor for Base class

    Base(int value)
    {

        baseValue = value;

        cout << "Base constructor invoked with value: " <<
baseValue << endl;

    }

};

// Derived class inheriting from Base class

class Derived : public Base
{
public:

    int derivedValue;

    // Parameterized constructor for Derived class

    Derived(int baseVal, int derivedVal) : Base(baseVal)
```

```

    {

        derivedValue = derivedVal;

        cout << "Derived constructor invoked with values: " <<
baseValue << ", " << derivedValue << endl;

    }

};

int main()
{

    Derived derivedObj(10, 20);

    return 0;

}

```

2.d) Is there any error in this code? if yes correct the code display the output

```

#include <iostream>

```

```

using namespace std;

class A
{
public:

    void cheers()

    {

```



```
        cout << "Class A: Hip-hip-hooray" << endl;

        ;

    }

};

class B
{
public:

    void cheers()

    {

        cout << "Class B: Hip-hip-hooray" << endl;

        ;

    }

};

class C : public A, public B
{

};

int main()

{

    C obc;
```

```
    obc.cheers();  
}
```

ANS: there is an ambiguity when you call the `cheers()` function on an instance of class `C`. The compiler doesn't know whether to call the `cheers()` function from class `A` or class `B`.

To resolve this ambiguity, we can explicitly specify which version of the function you want to call using the scope resolution operator `::`. Here's the corrected code:

```
#include <iostream>  
  
using namespace std;  
  
class A  
{  
public:  
    void cheers()  
    {  
        cout << "Class A: Hip-hip-hooray" << endl;  
    };  
};  
  
class B  
{  
public:  
    void cheers()
```

```

    {

        cout << "Class B: Hip-hip-hooray" << endl;

        ;

    }

};

class C : public A, public B
{
public:

    void cheers()

    {

        A::cheers(); // Call cheers() from class A

        B::cheers(); // Call cheers() from class B

    }

};

int main()

{

    C obc;

    obc.cheers();

}

```

OUTPUT

Class A: Hip-hip-hooray

Class B: Hip-hip-hooray

3.a) What is virtual function? Explain with example.

ANS : A virtual function is a member function in a base class that is declared with the **virtual** keyword. It allows the function to be overridden in derived classes.

```
#include <iostream>

using namespace std;

class Base
{
public:
    virtual void display()
    {
        cout << "Base class" << endl;
    }
};

class Derived : public Base
{
public:
    void display()
    {
        cout << "Derived class" << endl;
    }
}
```

```

    }

};

int main()
{
    Base *basePtr;

    Derived derivedObj;

    basePtr = &derivedObj; // Assigning address of Derived
object to base class pointer

    // Call virtual function through the base class pointer

    basePtr->display();

    return 0;
}

```

3.b) What do you know about early binding and late binding? Discuss the pros and cons

ANS: Early binding and late binding are two terms associated with the concept of function binding in programming languages.

Early Binding (Static Binding): Early binding refers to the process of associating a function call with its corresponding function implementation at compile-time. It is also known as static binding because the binding is determined statically, based on the type of the variable or reference.

Pros of Early Binding:

1. **Efficiency:** Early binding is generally faster because the function call is resolved at compile-time, eliminating the need for runtime lookup.
2. **Predictability:** The behavior of early-bound function calls is determined at compile-time, making the program's flow more predictable and easier to reason about.
3. **Compiler Optimization:** Early binding allows compilers to perform optimizations like inlining, constant folding, and function specialization, which can improve performance.

Cons of Early Binding:

1. **Lack of Polymorphism:** Early binding does not support polymorphism or dynamic dispatch. The function call is bound to the function implementation based on the static type of the variable or reference, which can limit the flexibility and extensibility of the code.
2. **Inflexibility:** With early binding, it is not possible to change the behavior of a function call based on the runtime type of the object. This can be a limitation in certain scenarios, such as implementing runtime polymorphism and implementing certain design patterns like the Factory Pattern or Strategy Pattern.

Late Binding (Dynamic Binding): Late binding refers to the process of associating a function call with its corresponding function implementation at runtime. The actual function to be called is determined dynamically based on the type of the object being referenced or pointed to.

Pros of Late Binding:

1. **Polymorphism:** Late binding enables polymorphism, allowing different derived classes to provide their own implementations of the same function. This promotes code reuse, extensibility, and flexibility.
2. **Run-time Adaptability:** Late binding allows changing the behavior of function calls based on the runtime type of the object. This enables runtime polymorphism and facilitates dynamic behavior adaptation.

3. Dynamic Dispatch: Late binding enables dynamic dispatch, where the appropriate function implementation is resolved at runtime based on the actual type of the object.

Cons of Late Binding:

1. Overhead: Late binding involves runtime lookup and resolution, which can introduce a slight performance overhead compared to early binding.
2. Reduced Predictability: The behavior of late-bound function calls can only be determined at runtime, which may make the program's flow less predictable and harder to reason about.
3. Potential Errors: Late binding relies on the correct setup and usage of the virtual and override keywords to ensure proper functioning. Improper use of these keywords can lead to errors or unexpected behavior.

Or

When a virtual function is called pure virtual? Write an appropriate program to make your understanding clear.

ANS: a virtual function is a member function of a base class that can be overridden in derived classes. When a virtual function is declared as "pure virtual," it means that it has no implementation in the base class and must be overridden in the derived classes.

```
#include <iostream>

using namespace std;

class Shape {
public:
    virtual double calculateArea() = 0; // Pure virtual function
};
```

```
class Rectangle : public Shape {  
private:  
    double width;  
    double height;  
  
public:  
    Rectangle(double w, double h) : width(w), height(h) {}  
  
    double calculateArea() override {  
        return width * height;  
    }  
};
```

```
class Circle : public Shape {  
private:  
    double radius;  
  
public:  
    Circle(double r) : radius(r) {}  
  
    double calculateArea() override {  
        return 3.14159 * radius * radius;  
    }  
};
```



```

    }

};

int main() {

    Rectangle rectangle(4.0, 5.0);

    Circle circle(3.0);

    // Accessing the calculateArea() function through base
    class pointers

    Shape* shape1 = &rectangle;

    Shape* shape2 = &circle;

    cout << "Area of rectangle: " << shape1->calculateArea()
    << endl;

    cout << "Area of circle: " << shape2->calculateArea() <<
    endl;

    return 0;

}

```

4.a) What is an exception? What are the advantages of using exception handling mechanism in a program?

An exception is an event that occurs during the execution of a program and disrupts the normal flow of instructions. It is usually an error condition or an exceptional situation that requires special handling. When an exception occurs, the program transfers control to a specific block of code called an

exception handler, which handles the exception and provides a way to recover from the error or exceptional situation.

The advantages of using exception handling mechanism in a program include:

1. **Separation of Error Handling Code:** Exception handling allows you to separate the error handling code from the normal flow of the program. This improves the readability and maintainability of the code by keeping the error handling logic separate from the core business logic.
2. **Centralized Error Handling:** With exception handling, you can centralize the error handling code in one or more exception handlers. This makes it easier to manage and maintain error handling logic since you don't need to scatter error checking and recovery code throughout your program.
3. **Graceful Error Handling:** Exception handling provides a way to gracefully handle errors and exceptional situations. Instead of abruptly terminating the program or causing it to crash.
4. **Robustness and Reliability:** Exception handling enhances the robustness and reliability of the program by providing a mechanism to handle unexpected or exceptional conditions. It helps prevent program crashes and allows you to handle various types of errors, such as runtime errors, input validation errors, and resource allocation failures, without disrupting the normal execution flow.
5. **Maintain Program Flow:** By catching and handling exceptions, you can maintain the flow of the program even when errors occur. This can be particularly useful in situations where you need to recover from errors and continue program execution or perform alternate actions.

Overall, exception handling provides a structured and controlled approach to dealing with errors and exceptional situations, promoting code maintainability, error recovery, and program reliability.

or, Show the general form of try, catch and throw for exception handling. In your own words describe their operations.

ANS: Exception handling in C++ typically involves three main components: **try**, **catch**, and **throw**. Here's the general form and a description of their operations:

try: The `try` block is used to enclose the code that may potentially throw an exception. It is followed by one or more `catch` blocks to handle specific types of exceptions. The general form of the `try` block is:

```
try {  
    // Code that may throw an exception  
} catch (ExceptionType1 ex1) {  
    // Exception handling code for ExceptionType1  
} catch (ExceptionType2 ex2) {  
    // Exception handling code for ExceptionType2  
} catch (...) {  
    // Default exception handling code for any other exception  
}
```

Inside the `try` block, you write the code that may throw an exception. If an exception is thrown within the `try` block, the normal flow of execution is interrupted, and the program looks for an appropriate `catch` block to handle the exception.

catch: The `catch` block is used to catch and handle specific types of exceptions. It allows you to specify the type of exception you want to handle and provides a block of code to execute when that specific exception is thrown. The general form of a `catch` block is:

```
catch (ExceptionType ex) {  
    // Exception handling code  
}
```

Inside the `catch` block, you write the code to handle the exception of type `ExceptionType`. You can access information about the exception, such as error messages or specific data, using the exception object `ex`.

You can have multiple **catch** blocks following a **try** block to handle different types of exceptions.

throw: The **throw** statement is used to explicitly throw an exception. It is typically used within a **try** block when an error or exceptional condition occurs. The general form of the **throw** statement is:

```
throw ExceptionType(argument);
```

Here, **ExceptionType** specifies the type of exception you want to throw, and **argument** is an optional value or object that provides additional information about the exception. When the **throw** statement is encountered, the program immediately exits the current block of code and starts searching for an appropriate **catch** block to handle the exception.

4b) Write a generic function, called min (), that returns the lesser of its two arguments.

```
#include <iostream>

template <typename T>

T min(const T &a, const T &b)

{

    return (a < b) ? a : b;

}

int main()

{

    int num1 = 10;
```

```

    int num2 = 5;

    std::cout << "The lesser number is: " << min(num1,
num2) << std::endl;

    double value1 = 3.14;

    double value2 = 2.71;

    std::cout << "The lesser value is: " <<
min<double>(value1, value2) << std::endl;

    return 0;
}

```

4c) What is generic function and generic class?

Generic functions and generic classes are constructs that allow you to write code that can work with multiple types without having to duplicate the code for each specific type. This enables code reuse and flexibility by providing a template that can be instantiated with different types.

Generic Function: A generic function is a function that can operate on different types of arguments. It is defined using templates and allows you to write code that is independent of the specific type.

```

#include <iostream>

```

```

template<typename T>

void print(T value) {

    std::cout << value << std::endl;

}

int main() {

    print(10);                // Prints: 10

    print("Hello, World!");   // Prints: Hello, World!

    print(3.14);              // Prints: 3.14

    return 0;

}

```

In this example, the `print()` function is defined as a generic function using the template keyword. The function takes a single argument `value` of type `T`, which can be any type. Inside the function, we simply print the value.

Generic Class: A generic class, also known as a template class, is a class that can be instantiated with different types. It provides a blueprint for creating objects of various types. Here's an example

```

#include <iostream>

template<typename T>

class MyContainer {

public:

    void insert(T value) {

        // Insert the value into the container

    }

}

```

```

    T get() {

        // Retrieve a value from the container

    }

};

int main() {

    MyContainer<int> intContainer;

    intContainer.insert(10);

    int value = intContainer.get();    // Retrieves an integer

    MyContainer<double> doubleContainer;

    doubleContainer.insert(3.14);

    double result = doubleContainer.get();    // Retrieves a
double

    return 0;

}

```

In this example, the `MyContainer` class is defined as a generic class using the template keyword and a type parameter `T`. The class provides two member functions: `insert()` and `get()`. The `insert()` function takes a value of type `T` and inserts it into the container. The `get()` function retrieves a value of type `T` from the container.

Using generic classes allows us to define a single class template that can work with different types, promoting code reuse and flexibility.

Both generic functions and generic classes in C++ provide powerful mechanisms for writing code that is more flexible and reusable across different types.

Or What is STL? Define a container, an iterator and an algorithm as they relate to the STL

ANS: STL stands for the Standard Template Library. It is a powerful library in C++ that provides a collection of generic algorithms and data structures as templates. The STL allows programmers to write efficient and reusable code by providing a set of standardized components that can be used with various data types.

Container: In the context of the STL, a container is a data structure that stores and organizes a collection of elements. It provides a way to store, access, and manipulate the elements. Containers in the STL include vectors, lists, sets, maps, stacks, queues, and more. Containers provide different characteristics, such as dynamic resizing, ordered or unordered storage, and fast insertion or retrieval, depending on the specific container type.

Example:

```
#include <iostream>

#include <vector>

int main() {

    std::vector<int> numbers;    // Vector container

    numbers.push_back(10);

    numbers.push_back(20);

    numbers.push_back(30);

    for (const auto& number : numbers) {
```



```

        std::cout << number << " "; // Prints: 10 20 30

    }

    return 0;
}

```

Iterator: An iterator is an object that provides a way to traverse the elements of a container. It acts as a general-purpose pointer that points to elements within the container.

```

#include <iostream>

#include <vector>

using namespace std;

int main()
{
    vector<int> numbers = {10, 20, 30};

    vector<int>::iterator it; // Iterator

    for (it = numbers.begin(); it != numbers.end(); ++it)
    {
        cout << *it << " "; // Prints: 10 20 30
    }
}

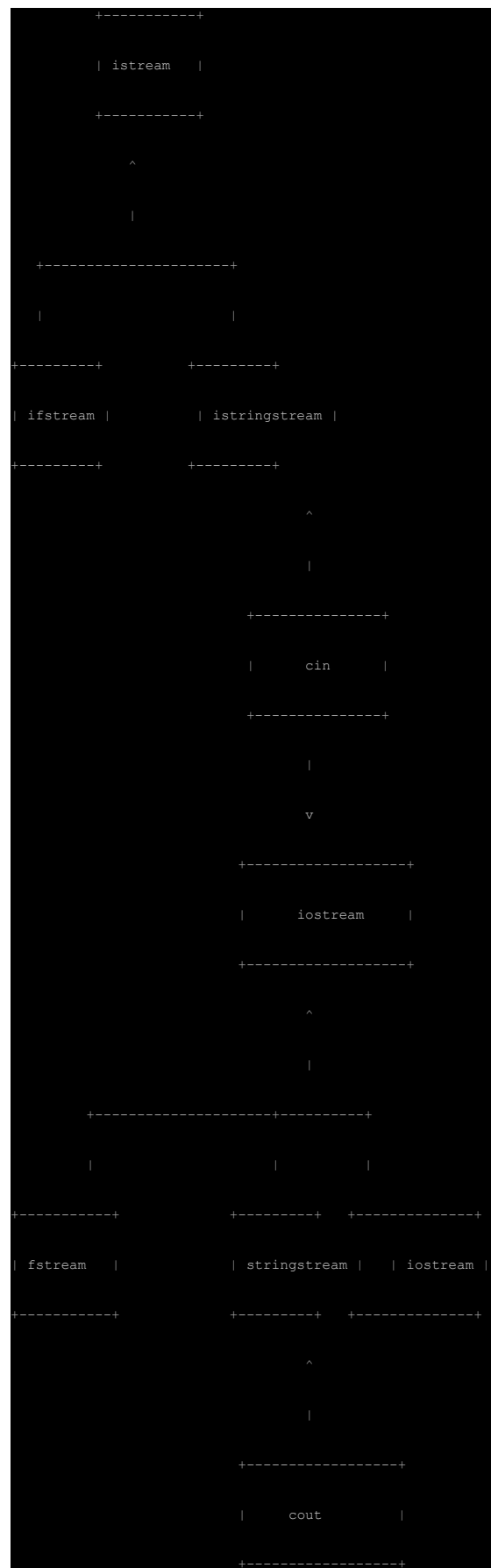
```

```
return 0;  
  
}
```

5. a) What is a stream? Write stream classes hierarchy for console I/O operations.

ANS: In C++, a stream is an abstraction that represents a sequence of characters that can be read from or written to. Streams provide a standardized way to perform input and output (I/O) operations, allowing data to be transferred between a program and various input/output devices, such as the console, files, or network.

The stream classes hierarchy for console I/O operations is illustrated below



or, What is manipulator? Formulate the differences between manipulators and ios member functions.

ANS: In C++, a manipulator is a special type of function or object that is used to modify the behavior of input/output streams. Manipulators are typically used in conjunction with the insertion (<<) and extraction (>>) operators to modify the formatting and state of the stream.

ere are the key differences between manipulators and **ios** member functions:

1. Syntax and Usage:

- Manipulators are functions or objects that are invoked using the insertion (<<) or extraction (>>) operators in the stream expression.
- For example, `cout << setw(10) << intValue;` uses the `setw()` manipulator to set the field width of the output stream.
- `ios` member functions, on the other hand, are called directly on the stream object using the dot operator.
- For example, `cout.width(10);` sets the width of the output stream directly using the `width()` member function.

2. Scoping:

- Manipulators are typically global in scope and can be used with any input/output stream.
- They can be defined by the programmer or provided by the C++ Standard Library.
- For example, `setw()` and `setfill()` are manipulators that are defined in the `<iomanip>` header and can be used with any stream.
- `ios` member functions, on the other hand, are specific to a particular stream object.
- They are part of the `ios` class hierarchy and can only be called on instances of stream classes such as `iostream`, `istream`, or `ostream`.

5.b) Write a program that implements the following ios functions: width(), precision(), fill(), setf()

```
#include <iostream>

#include <iomanip>

using namespace std;

int main()
{
    int intValue = 12345;

    double doubleValue = 3.14159;

    // width() - Set the field width for the next
    output operation

    cout << setw(10) << intValue << endl;    //
    Prints:      12345

    cout << setw(10) << doubleValue << endl; //
    Prints:     3.14159

    // precision() - Set the precision for
    floating-point numbers

    cout << setprecision(4) << doubleValue << endl;
    // Prints: 3.142
```

```

    // fill() - Set the fill character for the next
output operation

    cout << setfill('*') << setw(10) << intValue <<
endl; // Prints:*****12345

    // setf() - Set the formatting flags

    cout.setf(ios::scientific | ios::uppercase);

    cout << doubleValue << endl; // Prints:
3.141590E+00

    return 0;
}

```

5.C) Design a program to write the following information to a file called WhoAreYou.txt:

Name: xxxxxxxx

Semester: Spring 2022

Course Code: CSE-1221

Course Title: Computer Programming 2

ANS:

```
#include <iostream>

#include <fstream>

using namespace std;


int main()
{
    // Open the file

    ofstream outputFile("WhoAreYou.txt");

    // Check if the file was successfully opened
    if (!outputFile)
    {
        cerr << "Error opening the file." << endl;
        return 1;
    }

    // Write the information to the file

    outputFile << "Name: xxxxxxxx\n";

    outputFile << "Semester: Spring 2022\n";

    outputFile << "Course Code: CSE-1221\n";
```

```
    outputFile << "Course Title: Computer  
Programming 2\n";  
  
    // Close the file  
    outputFile.close();  
  
    cout << "Data written to the file successfully."  
<< endl;  
  
    return 0;  
}
```